TSR-ECHO Sonar Configuration Web Page

Overview

The **TSR-ECHO Sonar Configuration** web page is a portable browser-based utility for viewing and modifying configuration parameters in the SimpleSonar TSR-ECHO firmware.

It communicates directly with the sonar over a serial connection using the browser's **Web Serial API**. It is intended for Windows, Linux, and macOS using a Chromium-based browser that supports Web Serial. No Python application or platform-specific executable is required.

Files

The Version 1.1 distribution contains:

- `TSR-ECHO-Config.html` — main user interface, serial communications, and C2/C3 protocol handling.
- `parameters.js` — definitions controlling how firmware parameters are displayed and edited.
- `serialports.js` — USB serial adapter vendor filters.
- `logo.png` — optional product/company logo.
- `README.txt` — brief usage notes.

The parameter definitions and serial-port filters are deliberately separate from the main HTML application so they can be changed without modifying the communications code.

Starting the Utility

Open `TSR-ECHO-Config.html` in a Chromium-based browser supporting the Web Serial API.

Select the required baud rate and press **Connect**. The browser displays its serial-port selection dialog. After a port is selected, the page opens the port and automatically requests the current TSR-ECHO configuration.

The default baud rate is **115200 baud**. The last selected baud rate is stored in browser local storage and restored the next time the utility is opened.

Serial Port Selection

Web browsers do not expose operating-system serial device names directly to JavaScript. The application therefore cannot explicitly filter Linux devices such as `/dev/ttyS*`, `/dev/ttyUSB*`, or `/dev/ttyACM*`.

Instead, `serialports.js` contains USB vendor filters. Version 1.1 ships with:

```
const serialPortFilters = [
  { usbVendorId: 0x0403 },  // FTDI
  { usbVendorId: 0x10C4 }   // Silicon Labs
];
```

Additional USB adapter vendors can be added when required. To disable filtering and let the browser show all serial devices:

```
const serialPortFilters = [];
```

The browser controls the final device-selection dialog and serial access permissions.

Reading the Configuration

Immediately after connection, or when **Refresh (C3)** is pressed, the application sends:

```
C3<CR><LF>
```

The TSR-ECHO responds with a JSON object containing the currently available configuration and status parameters.

Serial data may arrive in several separate USB/serial reads. The application accumulates the data until a complete JSON object is available.

The current firmware response is terminated by a NUL character. The next command prompt follows as:

```
<CR><LF>+
```

The application treats NUL as the end of the response and ignores the following command prompt.

Parameter Definitions

The main HTML does not hard-code how each sonar parameter is presented. `parameters.js` describes each known parameter. For example:

```
PingClockFrequency: {
    label: "Transducer Frequency",
    category: "Sonar / Transducer",
    type: "integer",
    units: "Hz",
    editable: true,
    advanced: false,
    help: "Operating frequency of the sonar transducer."
}
```

Definitions can specify the display label, category, data type, units, editability, advanced/hidden status, minimum and maximum values, step size, display

scaling, selection-list values, and help text.

Only parameters actually returned by the connected firmware are rendered.

Parameter Categories

Version 1.1 uses categories including:

- Sonar / Transducer
- Ping / Range
- Gain / Processing
- Rotator
- Communications
- Operating Mode
- Device
- Status
- System / Advanced

Only categories containing returned parameters are displayed.

Normal, Advanced, and Hidden Parameters

A normal parameter is displayed whenever it is returned by the firmware.

A parameter with:

```
advanced: true
```

is shown only when **Show advanced settings** is enabled.

A parameter with:

```
hidden: true
```

is never displayed. This is intended primarily for internal or firmware-derived values.

Visibility and editability are independent. A visible parameter can still be read-only by specifying:

```
editable: false
```

Unknown Firmware Parameters

If C3 returns a parameter that is not defined in `parameters.js`, the application does not fail or silently discard it.

Unknown parameters are placed in **Unknown / Firmware Parameters**, visible with **Show advanced settings**, and are read-only by default. This provides forward compatibility when firmware introduces a parameter before the web definitions have been updated.

Changing Parameters

When an editable control is changed, the application compares the new value with the value most recently received from the TSR-ECHO.

Only values that actually changed are recorded as pending changes. Modified fields are highlighted and the page displays the number of pending changes.

For example:

`C2,{"PingClockFrequency":400000,"RangeEnd":15000}<CR><LF>`

Only the changed parameters are included.

Applying Changes

Press **Apply Changes** to send pending parameters using:

`C2,{"parameter":value,...}<CR><LF>`

After sending C2, the application waits briefly for the firmware to apply and recalculate its settings, then automatically sends:

`C3<CR><LF>`

The resulting C3 response replaces the displayed values. This keeps the firmware authoritative for calculated and dependent parameters rather than duplicating those calculations in JavaScript.

For example:

1. The user changes `PingClockFrequency`.
2. The page sends the new value with C2.
3. Firmware recalculates dependent sampling and timer values.
4. The page sends C3.
5. The page displays the configuration actually established by firmware.

Reverting Changes

Revert discards all pending edits that have not been sent to the TSR-ECHO and restores the values from the most recent C3 response.

It does not send anything to the sonar.

Refreshing

Refresh (C3) requests the current configuration directly from the TSR-ECHO. A refresh replaces pending page edits with the values returned by the device.

Communications Log

The expandable **Communications Log** records significant serial activity, for example:

```
17:57:56 INFO: Serial port opened at 115200 baud
17:57:56 TX: C3<CR><LF>
17:57:56 RX: {"ClockFrequency":240000000,...}
```

Transmit entries explicitly show `<CR><LF>` to make command termination clear.

Disconnecting

Disconnect stops the serial read loop, releases the browser's serial reader and writer, and closes the selected serial port.

Derived Firmware Values

The TSR-ECHO firmware remains responsible for calculating hardware-dependent values. The web page should normally expose high-level configuration choices instead of low-level timer settings.

For example, changing `PingClockFrequency` may cause firmware to recalculate:

```
SamplingFrequency
SamplingPrescaler
SamplingPulseLength
SamplingDutyCycle
PingClockPulseLength
PingClockDutyCycle
```

These values can be marked `hidden: true` in `parameters.js`.

This prevents the web configurator and firmware from establishing inconsistent configurations.

Speed of Sound Field Name

The current C3 response uses:

`SpeendOfSound`

Version 1.1 intentionally uses that spelling internally so it matches the current firmware protocol. The user-facing label remains **Speed of Sound**.

When firmware is corrected to `SpeedOfSound`, the corresponding key in `parameters.js` should also be changed.

Browser Requirements

The utility depends on the Web Serial API and should be used with a Chromium-based browser that supports it.

Serial access requires the user to select and authorize a device through the browser. Browser and operating-system security policies determine whether Web Serial is available when the HTML file is opened directly from the local filesystem.

Design Structure

The configuration system is intentionally divided into three main components:

```
TSR-ECHO-Config.html
    User interface
    Web Serial communications
    C2/C3 protocol
    Change tracking
    JSON processing

parameters.js
    Parameter names
    Categories
    Control types
    Visibility
    Editability
    Units and validation

serialports.js
    Permitted USB serial adapter vendors
```

This separation allows firmware parameters and supported USB adapters to evolve without substantial changes to the main application.